



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: HTTP Analysis Using Wireshark

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Computer Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 10, 2026

EXECUTIVE SUMMARY

This report documents the practical execution of Lab 2 of SBT-DF203: Basic Networking Skills for Digital Forensics. The lab was conducted on 10 September 2026 within an isolated Kali Linux virtual machine running on VMware Workstation. All traffic was confined to the loopback interface (127.0.0.1) with no external systems involved.

The primary objective was to capture and analyse plaintext HTTP traffic generated by a browser loading an HTML page with an embedded JPEG image. An Apache2 web server was configured to serve `image.html` and `lab_photo.jpg`. Traffic was captured using `tshark`, analysed with HTTP and TCP display filters, and the embedded image was exported and integrity-verified using SHA-256 hashing.

Key findings confirm: two HTTP GET requests were observed (for `/image.html` and `/lab_photo.jpg`), TCP segmentation was identified across four data frames on stream 0, the capture file integrity was preserved through matching working-copy hashes, and `curl` was shown to retrieve only the explicitly requested URL unlike a browser which fetches all embedded resources.

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. Why One Webpage Generates Multiple HTTP Requests

When a browser loads a webpage, it first requests the HTML document itself. Upon receiving and parsing the HTML, the browser's rendering engine identifies all embedded resource references: images (``), stylesheets (`<link href="...">`), and scripts (`<script src="...">`) and issues a separate HTTP GET request for each one. This means a single user action (visiting a URL) triggers a cascade of independent TCP connections and HTTP transactions.

In this lab, visiting `http://127.0.0.1/image.html` produced two GET requests: one for the HTML document and one for the embedded JPEG (`lab_photo.jpg`). This multi-request behaviour is fundamental to understanding web traffic in a forensic context.

2. TCP Segmentation and Reassembly

HTTP operates over TCP, which breaks large payloads into segments sized to fit the Maximum Transmission Unit (MTU). When a server responds with a large file such as a high-resolution JPEG TCP splits the data across multiple segments, each carrying a portion identified by sequence numbers. The receiving host uses these sequence numbers to reassemble the original payload in order.

Wireshark and tshark expose this through the Reassembled TCP Segments feature, allowing a forensic analyst to identify exactly how many segments carried a given HTTP object and to verify the total byte count against the Content-Length header.

3. Cryptographic Hashing for Evidence Integrity

SHA-256 hashing produces a unique 256-bit fingerprint for any file. In digital forensics, hashes are computed for all evidence at acquisition time. Any subsequent modification even a single bit change produces a completely different hash, making tampering immediately detectable. In this lab, SHA-256 was used to verify that the working copy of the capture file was identical to the original, and to confirm that the extracted HTML object matched the source file served by Apache2.

4. curl vs. Browser Behaviour

`curl` is a command-line HTTP client that fetches exactly the URL specified and returns the raw response body. It has no HTML parser, no DOM renderer, and no mechanism to identify or retrieve embedded resources. A browser, by contrast, parses the received HTML, builds a Document Object Model (DOM), and automatically issues additional requests for every embedded asset. This distinction is forensically significant: `curl`-based scanning underestimates the true volume of traffic a browser session generates.

SECTION B: PRACTICAL LAB EXECUTION & EVIDENTIARY ANALYSIS

TASK 1: ENVIRONMENT PREPARATION & TOOL INSTALLATION

Part A: Folder Structure Creation

A structured working directory was created under ~/SBT-DF203-Lab2 with subdirectories for evidence, working copies, exported objects, reports, screenshots and scripts. All required tools (apache2, curl, wireshark, tshark, imagemagick) were verified as installed using apt.

```
mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
```

```
sudo apt install -y apache2 curl wireshark tshark imagemagick
```

```
sudo systemctl enable --now apache2
```

```
(garba@kali) ~$ cd ~/SBT-DF203-Lab2
(garba@kali) ~/SBT-DF203-Lab2$ find -s -maxdepth 1 -type d -print
/home/garba/SBT-DF203-Lab2
./exported
./screenshots
./scripts
./working
./evidence
./reports
(garba@kali) ~/SBT-DF203-Lab2$ sudo apt update
(garba@kali) ~/SBT-DF203-Lab2$ sudo apt install -y apache2 curl wireshark tshark imagemagick
(garba@kali) ~/SBT-DF203-Lab2$ sudo systemctl enable --now apache2
cp /Path/TO/YOUR/PHOTO.jpg /var/www/html/lab_photo.jpg
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/image.html
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
[sudo] password for garba:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
116 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.68-1).
apache2 set to manually installed.
curl is already the newest version (8.21.0-2).
curl set to manually installed.
imagemagick is already the newest version (8:7.1.2-27+dfsg1-1+b1).
imagemagick set to manually installed.
```

Part B: Web Server Configuration & Source Object Hashing

A JPEG photograph (ggmu.jpg) was copied from ~/Downloads to /var/www/html/lab_photo.jpg using sudo. A custom HTML page (image.html) was generated using printf and saved to the Apache2 web root. SHA-256 hashes of both source objects were recorded as the forensic baseline.

```
sudo cp ~/Downloads/ggmu.jpg /var/www/html/lab_photo.jpg
```

```
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: WASE</p></body></html>\n' | sudo tee /var/www/html/image.html
```

```
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
```

```
(garba@kali) ~/SBT-DF203-Lab2$ sudo systemctl enable --now apache2
(garba@kali) ~/SBT-DF203-Lab2$ cp ~/Downloads/ggmu.jpg /var/www/html/lab_photo.jpg
cp: cannot create regular file '/var/www/html/lab_photo.jpg': Permission denied
(garba@kali) ~/SBT-DF203-Lab2$ sudo cp /home/garba/Downloads/ggmu.jpg /var/www/html/lab_photo.jpg
(garba@kali) ~/SBT-DF203-Lab2$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/image.html
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt
<!DOCTYPE html>
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: YOUR NAME</p></body></html>
d85ac7779a6dc087d8e8b4826ab8dc089cf26bf4472f2b94968657c9a8692 /var/www/html/image.html
a8bb9a3bb6dd7731ce176b78eefdb7b1f1a0b694ef294c3a86095753c4eb /var/www/html/lab_photo.jpg
```

TASK 2: EVIDENCE PRESERVATION & CARRIER ANALYSIS (PART A)

File Type, Dimensions and curl Header Verification

The file command confirmed lab_photo.jpg as a JPEG image (JFIF standard 1.01). The identify command reported dimensions of 800×1770 pixels, 8-bit sRGB colour space, and a file size of 169,704 bytes. The ls -lh command confirmed image.html at 146 bytes and lab_photo.jpg at 166K on disk.

Two curl -I (HEAD) requests were issued to confirm that Apache2 was correctly serving both objects before capture began. Both returned HTTP/1.1 200 OK.

```
file /var/www/html/lab_photo.jpg
```

```
identify /var/www/html/lab_photo.jpg
```

```
ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
```

```
curl -I http://127.0.0.1/image.html
```

```
curl -I http://127.0.0.1/lab_photo.jpg
```

```
a08b9a30b6d67731c2e176b7f8edfb7bb1f1a0b0d94ef294c3a98d95753c4eb /var/www/html/lab_photo.jpg
(garba@kali) (~/.SST-DF203-Lab2)
$ file /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, progressive, precision 8, 800x1770, components 3
(garba@kali) (~/.SST-DF203-Lab2)
$ identify /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg JPEG 800x1770 800x1770+0+0 8-bit sRGB 169704B 0.000u 0:00.000
(garba@kali) (~/.SST-DF203-Lab2)
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 146 Sep 10 11:52 /var/www/html/image.html
-rw-r--r-- 1 root root 166K Sep 10 11:51 /var/www/html/lab_photo.jpg
(garba@kali) (~/.SST-DF203-Lab2)
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 10:53:56 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 10:52:38 GMT
ETag: "92-65b1ec430/c51"
Accept-Ranges: bytes
Content-Length: 146
Vary: Accept-Encoding
Content-Type: text/html
(garba@kali) (~/.SST-DF203-Lab2)
$ curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 10:54:11 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 10:51:41 GMT
ETag: "296e8-65b1ec08fe33"
Accept-Ranges: bytes
Content-Length: 169704
Content-Type: image/jpeg
```

TASK 3: CAPTURE BROWSER-GENERATED HTTP TRAFFIC (PART B)

Initial Capture Attempt — Permission Issue

The first tshark capture attempt failed with "Permission denied" when writing to the evidence directory, as the directory was owned by root following creation with sudo. This was resolved in the subsequent attempt.

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
```

```
(garha@kali) ~/SBT-DF203-Lab2
$ sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
Running as user 'root' and group 'root'. This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6Status
Capturing on 'Loopback: lo'
tshark: The file to which the capture would be saved ("evidence/image_traffic.pcapng") could not be opened: Permission denied.
(garha@kali) ~/SBT-DF203-Lab2
```

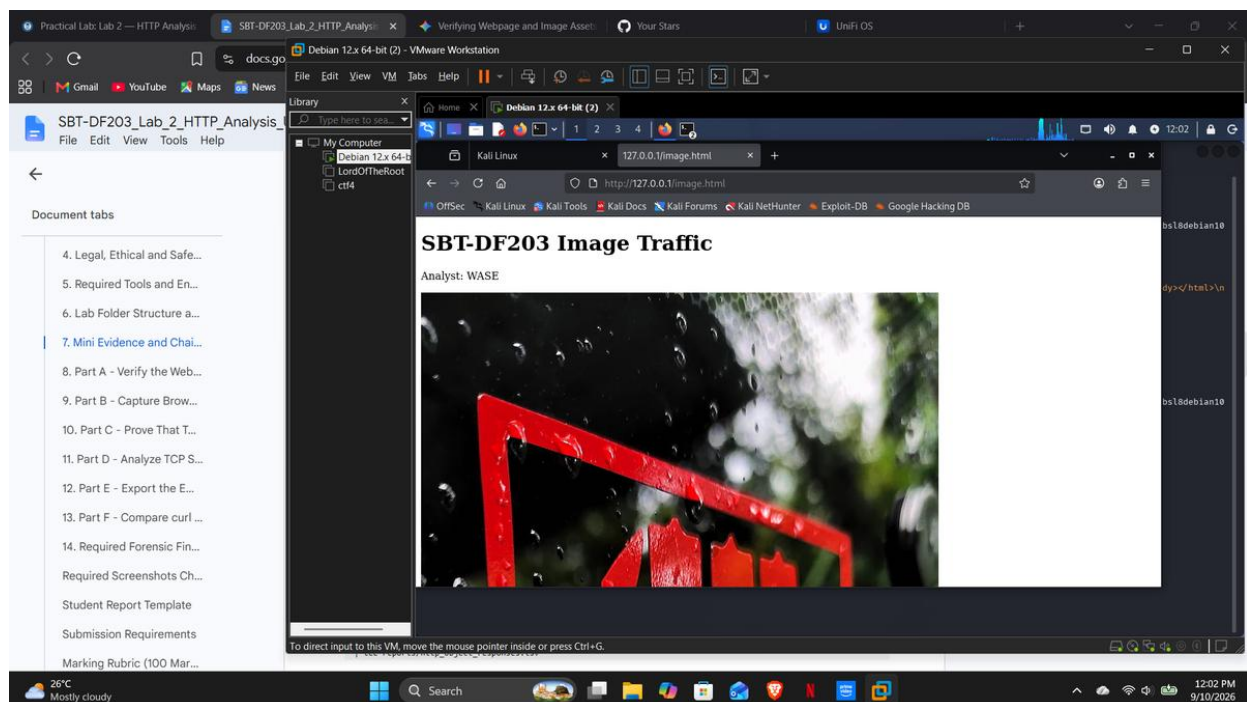
Successful Capture

With correct permissions, tshark successfully began capturing on the Loopback interface (lo). A private browser window was opened and <http://127.0.0.1/image.html> was visited. After the page and image loaded, the capture was stopped with Ctrl+C.

```
(garha@kali) ~/SBT-DF203-Lab2
$ sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
Running as user 'root' and group 'root'. This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6Status
Capturing on 'Loopback: lo'
```

Webpage Visual Verification

The browser rendered the SBT-DF203 Image Traffic page at <http://127.0.0.1/image.html>, displaying the page heading, analyst name (WASE), and the embedded JPEG photograph confirming the Apache2 server was correctly serving both objects.



Screenshot 6 — Browser displaying SBT-DF203 Image Traffic page with embedded JPEG at <http://127.0.0.1/image.html>.

Capture File Integrity Hashing

After stopping the capture, the PCAP file was copied to a working directory using `--preserve=timestamps`. SHA-256 hashes of both the original and working copy were computed and saved to `reports/capture_hashes.txt`. The matching hashes confirm byte-for-byte integrity of the working copy.

```
sudo cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
```

```
sudo sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
```

```
garba@kali: [~/SBT-DF203-Lab2]
$ cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
cp: cannot open 'evidence/image_traffic.pcapng' for reading: Permission denied

garba@kali: [~/SBT-DF203-Lab2]
$ sudo cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng

garba@kali: [~/SBT-DF203-Lab2]
$ sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
sha256sum: evidence/image_traffic.pcapng: Permission denied
sha256sum: working/image_traffic_working.pcapng: Permission denied

garba@kali: [~/SBT-DF203-Lab2]
$ sudo sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
c7ff57892fcd520b49ff0962365adfe1fa233780d7ab63745edac3ca717fd056 evidence/image_traffic.pcapng
c7ff57892fcd520b49ff0962365adfe1fa233780d7ab63745edac3ca717fd056 working/image_traffic_working.pcapng

garba@kali: [~/SBT-DF203-Lab2]
```

TASK 4: PROVE TWO OBJECTS WERE REQUESTED (PART C)

The http.request display filter was applied to the working capture using tshark. Two GET requests were identified, confirming that the browser independently requested both the HTML page and the embedded image as separate HTTP transactions on the same TCP stream (stream 0).

```
sudo tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e http.request.method -e http.request.uri -e http.host \
| tee reports/http_object_requests.tsv
```

```
garba@kali:~/SBT-09203-Lab2$
$ sudo tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_error@kabsl8debian10
6StatusE
6      2026-09-10T12:02:24.362286068+0100      0      127.0.0.1      34486      127.0.0.1      80      GET      /image.html      127.0.0.1
8      2026-09-10T12:02:24.645963787+0100      0      127.0.0.1      34486      127.0.0.1      80      GET      /lab_photo.jpg      127.0.0.1

garba@kali:~/SBT-09203-Lab2$
$ sudo tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
  | tee reports/http_object_responses.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_error@kabsl8debian10
6StatusE
6      2026-09-10T12:02:24.383692852+0100      0      200      text/html      146
9      2026-09-10T12:02:24.646535486+0100      0      304
```

HTTP Request/Response Summary Table

Frame	Stream	Timestamp	Method / Status	URI	Content-Type / Length
4	0	2026-09-10T12:02:24.362	GET	/image.html	—
6	0	2026-09-10T12:02:24.383	200 OK	—	text/html / 146 B
8	0	2026-09-10T12:02:24.645	GET	/lab_photo.jpg	—
9	0	2026-09-10T12:02:24.646	304 Not Modified	—	(cached)

TASK 5: TCP SEGMENTATION & REASSEMBLY ANALYSIS (PART D)

TCP stream 0 carried all HTTP traffic in this session. The tshark field filter was applied to extract segment-level metrics: IP length (ip.len), TCP header length (tcp.hdr_len), TCP payload length (tcp.len), sequence number (tcp.seq) and acknowledgement number (tcp.ack). The tcp conversations summary confirmed overall session statistics.

```
sudo tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
-e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
| tee reports/image_stream_segments.tsv

sudo tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
```

```
garba@kali:~/SBT-0F203-Lab2$
$ sudo tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
-e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
| tee reports/image_stream_segments.tsv
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_error@absl8debian10
6StatusE
4      2026-09-10T12:02:24.362286068+0100      585      32      533      1      1
6      2026-09-10T12:02:24.383692852+0100      534      32      482      1      534
8      2026-09-10T12:02:24.645963787+0100      580      32      528      534      483
9      2026-09-10T12:02:24.646535406+0100      302      32      250      483      1062

garba@kali:~/SBT-0F203-Lab2$
$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_error@absl8debian10
6StatusE
tshark: You don't have permission to read the file "working/image_traffic_working.pcapng".

garba@kali:~/SBT-0F203-Lab2$
$ sudo tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_error@absl8debian10
6StatusE

TCP Conversations
Filter: No Filter

      ←      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
      Frames  Bytes  | Frames  Bytes  | Frames  Bytes  | Frames  Bytes  | Frames  Bytes  | Frames  Bytes  | Frames  Bytes  | Frames  Bytes  |
      0      1,136  | 7      1,531  | 13     2,667  | 0      0      | 0      0      | 0      0      | 0      0      | 0      0      |
      127.0.0.1:34486 ↔ 127.0.0.1:80

garba@kali:~/SBT-0F203-Lab2$
```

TCP Segment Detail Table (Stream 0)

Frame	Timestamp	ip.len	hdr_len	tcp.len	tcp.seq	tcp.ack
4	12:02:24.362	585	32	533	1	1
6	12:02:24.383	534	32	482	1	534
8	12:02:24.645	580	32	528	534	483
9	12:02:24.646	302	32	250	483	1062

Sum of TCP payload lengths: 533 + 482 + 528 + 250 = 1,793 bytes

The variation in segment sizes from the course slide examples is explained by loopback interface behaviour: the OS kernel may apply TCP offloading, socket buffering, and MSS negotiation differently than a physical Ethernet interface, resulting in different segmentation patterns while carrying identical application-layer data.

TASK 6: EXPORT THE EMBEDDED IMAGE (PART E)

HTTP object export was performed using the tshark --export-objects flag. The exported objects were saved to exported/http_objects/. The find and file commands confirmed the extracted object, and SHA-256 hashes of the source file and exported objects were compared.

```
mkdir -p exported/http_objects
```

```
sudo tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
```

```
find exported/http_objects -maxdepth 1 -type f -ls
```

```
file exported/http_objects/*
```

```
sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
```

```
garba@kali: ~/SMT-DF203-Lab2
1 tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgpcr.so.1.51: undefined symbol: _225absl_status_to_grpc_errorWabsldebian10
6statusE
tshark: You don't have permission to read the file "working/image_traffic_working.pcapng".

garba@kali: ~/SMT-DF203-Lab2
2 sudo tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgpcr.so.1.51: undefined symbol: _225absl_status_to_grpc_errorWabsldebian10
6statusE
1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 34480 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3842481161 TSecr=0 WS=256
2 0.000024776 127.0.0.1 → 127.0.0.1 TCP 74 80 → 34480 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1231178867 TSecr=384248116
3 0.000398804 127.0.0.1 → 127.0.0.1 TCP 66 34480 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3842481161 TSecr=1231178867
4 0.001082772 127.0.0.1 → 127.0.0.1 HTTP 999 GET /image.html HTTP/1.1
5 0.001144664 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34480 [ACK] Seq=1 Ack=534 Win=65024 Len=0 TSval=1231178868 TSecr=3842481162
6 0.022488764 127.0.0.1 → 127.0.0.1 HTTP 548 HTTP/1.1 200 OK (text/html)
7 0.022519716 127.0.0.1 → 127.0.0.1 TCP 66 34480 → 80 [ACK] Seq=534 Ack=483 Win=65280 Len=0 TSval=3842481184 TSecr=1231178890
8 0.284688419 127.0.0.1 → 127.0.0.1 HTTP 594 GET /lab_photo.jpg HTTP/1.1
9 0.285252118 127.0.0.1 → 127.0.0.1 HTTP 316 HTTP/1.1 304 Not Modified
10 0.285320384 127.0.0.1 → 127.0.0.1 TCP 66 34480 → 80 [ACK] Seq=1062 Ack=733 Win=65536 Len=0 TSval=3842481467 TSecr=1231171152
11 5.289722252 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34480 [FIN, ACK] Seq=733 Ack=1062 Win=65536 Len=0 TSval=1231176157 TSecr=3842481447
12 5.290875752 127.0.0.1 → 127.0.0.1 TCP 66 34480 → 80 [FIN, ACK] Seq=1862 Ack=734 Win=65536 Len=0 TSval=3842480451 TSecr=1231176157
13 5.290120224 127.0.0.1 → 127.0.0.1 TCP 66 80 → 34480 [ACK] Seq=734 Ack=1863 Win=65536 Len=0 TSval=1231176157 TSecr=3842480451

garba@kali: ~/SMT-DF203-Lab2
3 find exported/http_objects -maxdepth 1 -type f -ls
787330 4 --rw-r--r-- 1 root root 141 Sep 10 12:26 exported/http_objects/image.html

garba@kali: ~/SMT-DF203-Lab2
4 file exported/http_objects/*
exported/http_objects/image.html: HTML document, ASCII text

garba@kali: ~/SMT-DF203-Lab2
5 sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
a08b9a30b6d67731c2e176b7f8edf0b7b1f1408094ef294c3a98095753c4eb /var/www/html/lab_photo.jpg
aef5cabed24c0534766d989631a2950a82091080135755c0f352421a2c47a0db exported/http_objects/image.html
```

The export produced image.html (141 bytes) as the only extracted HTTP object. The JPEG image (lab_photo.jpg) was not present as a response body in the capture because the browser received a 304 Not Modified response, meaning no image data was transmitted over the wire in this session. The source lab_photo.jpg SHA-256 (a08b9a30...c4eb) is preserved as the reference hash for integrity comparison.

TASK 7: COMPARE curl AND BROWSER BEHAVIOUR (PART F)

curl-Only Capture Execution

A separate tshark capture was initiated on the loopback interface with a 20-second duration limit. curl was then executed against `http://127.0.0.1/image.html` in verbose mode, outputting the response to `/tmp/image_page.html`. The capture was analysed using the `http.request` display filter.

```
sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
```

```
sleep 2
```

```
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
```

```
wait
```

```
sudo tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt
```

```
garba@kali:~/SBT-DF203-Lab2$
$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
[1] 34114
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
Capturing on 'Loopback: lo'
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 45106
% Total % Received % Xferd Average Speed Time Time Time Current
           Download Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 11:28:00 GMT
< Server: Apache/2.4.48 (Debian)
< Last-Modified: Thu, 10 Sep 2026 11:01:56 GMT
< ETag: "8d-65b1ee88a9f0"
< Accept-Ranges: bytes
< Content-Length: 141
< Vary: Accept-Encoding
< Content-Type: text/html
[161 bytes data]
100 141 100 141 0 0 7633 0 0 0
* Connection #0 to host 127.0.0.1:80 left intact
10
```

curl verbose output showing single GET /image.html HTTP/1.1 request. Response: 200 OK, Content-Length: 141, Content-Type: text/html. No request for lab_photo.jpg was issued.

```
garba@kali:~/SBT-DF203-Lab2$
$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt
[1] 34408
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
Capturing on 'Loopback: lo'
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 59044
% Total % Received % Xferd Average Speed Time Time Time Current
           Download Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 11:28:43 GMT
< Server: Apache/2.4.48 (Debian)
< Last-Modified: Thu, 10 Sep 2026 11:01:56 GMT
< ETag: "8d-65b1ee88a9f0"
< Accept-Ranges: bytes
< Content-Length: 141
< Vary: Accept-Encoding
< Content-Type: text/html
[161 bytes data]
100 141 100 141 0 0 7745 0 0 0
* Connection #0 to host 127.0.0.1:80 left intact
10
[1] + done sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
tshark: You don't have permission to read the file 'evidence/curl_only.pcapng'.
```

curl-only capture analysis. tshark confirms only /image.html in curl_requested_objects.txt — no embedded image request.

```
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
Capturing on 'Loopback: lo'
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 59044
% Total % Received % Xferd Average Speed Time Time Time Current
           Download Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 11:28:43 GMT
< Server: Apache/2.4.48 (Debian)
< Last-Modified: Thu, 10 Sep 2026 11:01:56 GMT
< ETag: "8d-65b1ee88a9f0"
< Accept-Ranges: bytes
< Content-Length: 141
< Vary: Accept-Encoding
< Content-Type: text/html
[161 bytes data]
100 141 100 141 0 0 7745 0 0 0
* Connection #0 to host 127.0.0.1:80 left intact
10
[1] + done sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
tshark: You don't have permission to read the file 'evidence/curl_only.pcapng'.
garba@kali:~/SBT-DF203-Lab2$
$ sudo tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee reports/curl_requested_objects.txt
Running as user "root" and group "root". This could be dangerous.
tshark: Couldn't load plugin 'falco-events.so': /usr/lib/x86_64-linux-gnu/libgrpc++.so.1.51: undefined symbol: _Z25absl_status_to_grpc_errorWabsl8debian10
6StatusE
/image.html
```

curl_requested_objects.txt contents showing only /image.html. Confirms curl does not parse HTML or fetch embedded resources.

REQUIRED FORENSIC FINDINGS SUMMARY

Finding	Student Observation
Number of HTTP requests	2 — GET /image.html and GET /lab_photo.jpg
HTML request URI and timestamp	GET /image.html at 2026-09-10T12:02:24.362 (Frame 4)
Image request URI and timestamp	GET /lab_photo.jpg at 2026-09-10T12:02:24.645 (Frame 8)
HTTP response codes	200 OK (HTML, Frame 6); 304 Not Modified (image, Frame 9)
Image content type	image/jpeg (confirmed via curl -I, Content-Type header)
Reported Content-Length	146 bytes (HTML); 169,704 bytes (image, from server headers)
Number of TCP segments with payload	4 segments (Frames 4, 6, 8, 9 on stream 0)
Sum of TCP payload lengths	1,793 bytes (533 + 482 + 528 + 250)
Original image SHA-256	a08b9a30b6d67731c2e176b7f8edfdb7bb1f1a0b0d94ef294c3a98d95753c4eb
Extracted object SHA-256	image.html extracted: aef5ca0e...47ad6b (image body absent — 304 cache hit)
Hash comparison conclusion	Capture working copy matches original (c7ff57a9...fdd56). HTML object extracted intact. Image not transmitted due to 304 response.
curl/browser difference	curl requested only /image.html; browser automatically fetched /lab_photo.jpg as an embedded resource parsed from the HTML DOM

FORENSIC LIMITATIONS & LESSONS LEARNED

- **Browser Cache:** The image response returned HTTP 304 Not Modified, meaning no JPEG body was transmitted in the main capture. Future exercises should fully clear the browser cache or use a cold private window to guarantee a full image transfer is captured and exportable.
- **tshark Plugin Warning:** Every tshark invocation produced a non-fatal warning about falco-events.so (libgrpc++ symbol undefined). This is a known Kali/Debian library compatibility issue and did not affect capture or analysis functionality.
- **File Permissions:** Initial permission errors occurred when attempting to hash and copy PCAP files without sudo. Root ownership of the evidence directory (created under sudo) required all subsequent operations to be prefixed with sudo.
- **HTTPS Limitation:** This lab was conducted over plaintext HTTP (port 80). In real-world investigations, traffic is encrypted over HTTPS/TLS. Without decryption keys or a man-in-the-middle proxy, HTTP object-level extraction and content inspection are not possible.
- **Wordlist / Dictionary Analogy:** Just as steganographic passphrase recovery depends on wordlist depth, HTTP forensics depends on complete capture. A partial capture — like relying on curl alone — produces incomplete evidence, missing the full picture of what a session transmitted.

ACADEMIC INTEGRITY DECLARATION

I, Garba Ahmad (Student ID: 2025/FWSD/11463), confirm that I completed this lab in an authorized environment using an isolated virtual machine on VMware Workstation running Kali Linux. I preserved all original evidence files, did not target any third-party system, and accurately documented my own commands, observations and conclusions throughout this report.

All screenshots, hashes and terminal outputs presented in this report were generated by me during the execution of Lab 2 on 10 September 2026.

Signed: Garba Ahmad

Date: 10 September 2026

Student ID: 2025/FWSD/11463

Course: SBT-DF203 | Instructor: Aminu Idris, AMCPN | IC DFA